Modul Praktikum 5

ABSTRACT CLASS & INTERFACE

D3 Teknik Informatika

POLITEKNIK NEGERI BANDUNG

Daftar Isi

Daftar Isi..... 2
1. Modul Overview..... 3
2. Persiapan Praktikum 4
3. Praktikum I - Employee, Manager, dan Sortable .....
5 4. Praktikum II - Goods dan Taxable
..... 9 5. Laporan dan
Pengumpulan..... 13 6. Checkpoint
dan Refleksi..... 14

1. Modul Overview

Pada pertemuan ini Anda akan melanjutkan pengalaman inheritance dari praktikum sebelumnya menuju dua kebutuhan baru: class yang berguna sebagai dasar hierarchy tetapi tidak semestinya dibuat sebagai object langsung, dan kontrak behavior yang dapat dipenuhi oleh class tanpa menambah parent class baru. Anda akan memahami keduanya melalui compile, run, error, dan perubahan output sebelum merapikan istilah teorinya.

1.1 Tujuan Pembelajaran

• Menjelaskan kembali hubungan superclass-subclass melalui kasus Employee dan Manager. • Membuat abstract class dan membuktikan bahwa abstract class tidak dapat diinstansiasi. •

Mendeklarasikan abstract method dan mengimplementasikannya pada subclass. • Menguji inheritance chain: Manager tetap membawa behavior yang diwarisi Employee dari Sortable. • Membuktikan bahwa Java class tidak dapat extends lebih dari satu class.
• Membuat interface sebagai kontrak behavior menggunakan kasus Taxable.
• Menggabungkan extends dan implements pada class yang sama.
• Menjaga encapsulation: state private tetap diakses melalui method yang disediakan class pemiliknya. • Membedakan fungsi abstract class dan interface berdasarkan code yang benar-benar dijalankan.

1.2 Cara Kerja di Setiap Langkah

PREDICT -> CODE -> RUN -> OBSERVE -> EXPLAIN

Sebelum perubahan penting, tulis dugaan singkat. Setelah compile/run, simpan output atau pesan error. Jangan langsung memperbaiki error: gunakan error sebagai bukti untuk menjelaskan aturan Java yang sedang Anda pelajari.

1.3 Alokasi Waktu

• Blok I - Employee, Manager, dan Sortable: 100 menit.
• Blok II - Goods, Food, Toy, Book, dan Taxable: 100 menit.

1.4 Batas Praktikum

• Fokus wajib: inheritance chain, abstract class, abstract method, extends, implements, interface, dan contract.
• Polymorphism dibahas hanya sejauh diperlukan untuk menguji object melalui reference abstract class/interface; pembahasan mendalam dilakukan pada pertemuan berikutnya.
• Tidak menggunakan database, GUI, framework, atau library eksternal selain library Java standar yang dibutuhkan.
• Jangan mengubah field private menjadi public untuk memudahkan subclass.

1.5 Penggunaan AI

Diagnosis pertama harus dilakukan sendiri. AI boleh digunakan setelah Anda memiliki bukti compile/run untuk meminta hint atau penjelasan error. Jangan meminta AI menghasilkan seluruh task. Catat bantuan yang digunakan dan verifikasi kembali dengan menjalankan code.

2. Persiapan Praktikum

2.1 Struktur Project

```
praktikum5/
├── src/
│   ├── Sortable.java
│   ├── Employee.java
│   ├── Manager.java
```

```
│   ├── EmployeeTest.java
│   ├── Goods.java
│   ├── Food.java
│   ├── Taxable.java
│   ├── Toy.java
│   ├── Book.java
│   └── GoodsTest.java
└── laporan/
```

Gunakan package yang konsisten jika project Anda memakai package. Jangan mencampur file dengan package berbeda dalam satu folder source.

2.2 Checklist Awal

- JDK dan IDE dapat compile/run project Java sederhana.
- Siapkan project baru dan folder laporan untuk menyimpan output/error penting.
- Siapkan catatan singkat untuk prediksi dan observasi tiap checkpoint.
- Gunakan source code final praktikum sebelumnya hanya sebagai referensi konsep inheritance, bukan untuk disalin menjadi kasus baru.

Jangan mengejar output akhir secepat mungkin.

Nilai pembelajaran ada pada perubahan kecil, error yang Anda amati, dan kemampuan menjelaskan mengapa suatu class harus extends atau implements sesuatu.

3. Praktikum I - Employee, Manager, dan Sortable

Blok pertama membangun kembali inheritance dari Employee ke Manager, kemudian menambahkan sebuah abstract class bernama Sortable. Tujuannya adalah melihat bagaimana abstract method membentuk kewajiban pada subclass dan bagaimana inheritance chain bekerja tanpa menulis extends dua kali.

Step 1 - Bangun class Employee (15 menit)

Buat Employee.java dengan state private dan behavior berikut.

```
public class Employee {
private String name;
private double salary;
private int hireDay;
private int hireMonth;
private int hireYear;

public Employee(String name, double salary,
int day, int month, int year) {
this.name = name;
this.salary = salary;
this.hireDay = day;
this.hireMonth = month;
this.hireYear = year;
}

public void print() {
System.out.println(name + " " + salary + " " + hireYear());
}

public void raiseSalary(double byPercent) {
salary *= 1 + byPercent / 100;
}

public int hireYear() {
return hireYear;
}

public double getSalary() {
return salary;
}
}
```

Checkpoint 1

Mana state Employee? Mana behavior Employee? Mengapa salary tetap dibuat private walaupun nanti Employee akan mempunyai subclass?

Step 2 - Uji beberapa object Employee (10 menit)

```
public class EmployeeTest {
public static void main(String[] args) {
Employee[] staff = new Employee[3];

staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);
```



```
for (Employee e : staff) {
e.raiseSalary(5);
}

for (Employee e : staff) {
e.print();
}
}
```

```
}
```

- Run program dan simpan output sebelum/ sesudah kenaikan salary jika Anda menambahkan output pembanding.
- Pastikan ketiga object memiliki state yang berbeda.

Step 3 - Buat Manager sebagai subclass Employee (15 menit)

Manager adalah Employee yang memiliki state tambahan secretaryName dan aturan kenaikan salary yang berbeda. Gunakan library Java standar untuk mendapatkan tahun saat ini.

```
import java.util.Calendar;
import java.util.GregorianCalendar;

public class Manager extends Employee {
 private String secretaryName;

 public Manager(String name, double salary,
 int day, int month, int year) {
 super(name, salary, day, month, year);
 secretaryName = "";
 }

 @Override
 public void raiseSalary(double byPercent) {
 GregorianCalendar today = new GregorianCalendar();
 int currentYear = today.get(Calendar.YEAR);
 double bonus = 0.5 * (currentYear - hireYear());
 super.raiseSalary(byPercent + bonus);
 }

 public String getSecretaryName() {
 return secretaryName;
 }
 }
```

Checkpoint 2

Mengapa constructor Manager memanggil super(...)? Pada raiseSalary(), mengapa Manager tetap memanfaatkan super.raiseSalary(...) daripada menulis ulang seluruh proses perubahan salary?

Step 4 - Uji Employee dan Manager dalam satu program (10 menit)

```
 Employee[] staff = new Employee[3];
staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

for (Employee e : staff) {
 e.raiseSalary(5);
```



```
 }

for (Employee e : staff) {
 e.print();
 }
```

- Run dan amati apakah object Manager menjalankan aturan raiseSalary() yang sama dengan Employee biasa.

Checkpoint 3

Lengkapi kalimat dengan bahasa Anda: Manager adalah sebuah _____, sehingga object Manager dapat ditempatkan pada array Employee.

Step 5 - Buat abstract class Sortable (10 menit)

Sekarang kita membutuhkan konsep yang menyatakan bahwa object dapat dibandingkan. Buat Sortable.java berikut.

```
public abstract class Sortable {
 public abstract int compare(Sortable other);
}
```

- Coba tulis `Sortable s = new Sortable();` pada class test, lalu compile.
- Simpan pesan error dan kembalikan code test ke kondisi bersih.

Checkpoint 4

Sortable berguna sebagai class untuk menetapkan kewajiban compare(...), tetapi tidak memiliki bentuk object konkret yang perlu dibuat langsung. Apa bukti compiler yang Anda dapatkan?

Catatan tentang shell_sort(...)

Pada latihan asal, method static shell_sort(Sortable[] a) ditampilkan dengan body algoritma sebagai placeholder. Karena algoritma Shell Sort tidak diberikan dan bukan target konsep pertemuan ini, modul ini menguji contract compare(...) secara langsung. Jika dosen memberikan implementasi shell_sort(...), Anda boleh menambahkannya dan menguji staff sebagai perluasan.

Step 6 - Jadikan Employee subclass dari Sortable (15 menit)

Ubah deklarasi Employee lalu compile sebelum menambahkan method compare(...).

```
public class Employee extends Sortable {
// state, constructor, dan method Employee yang sudah dibuat
}
```

- Compile. Catat error yang muncul karena Employee belum memenuhi abstract method milik

Sortable. Setelah mengamati error, tambahkan implementasi berikut di Employee.

```
@Override
public int compare(Sortable other) {
 Employee otherEmployee = (Employee) other;
```



```
 if (getSalary() < otherEmployee.getSalary()) return -1;
 if (getSalary() > otherEmployee.getSalary()) return 1;
 return 0;
}
```

Checkpoint 5

Apa yang diwajibkan Sortable kepada Employee? Mana bagian yang hanya deklarasi pada superclass abstract, dan mana bagian yang menjadi implementasi konkret di Employee?

Step 7 - Uji compare(...) (10 menit)

```
System.out.println(staff[0].compare(staff[1]));
System.out.println(staff[1].compare(staff[2]));
System.out.println(staff[2].compare(staff[0]));
```

• Prediksi hasil -1, 0, atau 1 sebelum run.
• Run dan jelaskan arti setiap hasil berdasarkan salary object yang dibandingkan.

Step 8 - Uji inheritance chain dan batas extends (15 menit)

Manager sudah extends Employee, sedangkan Employee sudah extends Sortable. Sekarang lakukan dua eksperimen.

Eksperimen A - Coba deklarasi berikut hanya untuk melihat respons compiler:

```
// SALAH - hanya untuk eksperimen
public class Manager extends Employee extends Sortable {
}
```

• Compile, simpan error, lalu kembalikan deklarasi menjadi `Manager extends Employee`.

Eksperimen B - Tanpa menulis `extends Sortable` pada Manager, coba:

```
Manager m = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
Employee e = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);
System.out.println(m.compare(e));
```

Checkpoint 6

Mengapa Manager tetap memiliki compare(...) walaupun tidak pernah menulis `extends Sortable`? Gambarkan rantai class: Sortable <- Employee <- Manager.

4. Praktikum II - Goods dan Taxable

Blok kedua menggunakan hierarchy Goods untuk membedakan dua jenis hubungan: Food, Toy, dan Book adalah jenis Goods; sementara hanya Toy dan Book yang memiliki kontrak Taxable. Anda akan mengimplementasikan diagram tersebut secara bertahap dan menguji apa yang terjadi jika sebuah class

mengaku implements interface tetapi tidak memenuhi method yang dijanjikan.

Step 9 - Pahami model sebelum coding (10 menit)

```
Goods
- description
- price
+ display()

Food extends Goods
- calories

Toy extends Goods implements Taxable
- minimumAge

Book extends Goods implements Taxable
- author

Taxable
- taxRate = 0.06
+ calculateTax()
```

Sebelum coding, tulis prediksi singkat:

- State apa yang cukup umum sehingga berada di Goods?
- State apa yang hanya dimiliki Food, Toy, dan Book?
- Mengapa Taxable tidak diterapkan pada semua subclass Goods pada model ini?

Step 10 - Buat class Goods (15 menit)

```
public class Goods {
private String description;
private double price;

public Goods(String description, double price) {
this.description = description;
this.price = price;
}

public String getDescription() {
return description;
}

public double getPrice() {
return price;
}

public void display() {
System.out.println("Description : " + description);
System.out.println("Price : " + price);
}
}
```


Checkpoint 7

Mengapa description dan price diletakkan di Goods, bukan ditulis ulang di Food, Toy, dan Book?

Step 11 - Buat Food sebagai subclass Goods (10 menit)

```
public class Food extends Goods {
private int calories;

public Food(String description, double price, int calories) {
super(description, price);
this.calories = calories;
}

public int getCalories() {
return calories;
}

@Override
public void display() {
super.display();
System.out.println("Calories : " + calories);
}
}
```

• Buat satu object Food dan pastikan display() menampilkan state Goods dan state Food.

Step 12 - Buat interface Taxable (10 menit)

Buat kontrak Taxable berikut. Interface menetapkan bahwa class yang mengimplementasikannya harus menyediakan calculateTax().

```
public interface Taxable {
double taxRate = 0.06;
double calculateTax();
}
```

• Coba `Taxable t = new Taxable();` lalu compile. Simpan error dan hapus kembali baris tersebut.

Checkpoint 8

Apa persamaan hasil eksperimen `new Sortable()` dan `new Taxable()`? Apa perbedaan bentuk deklarasi Sortable dan Taxable?

Step 13 - Buat Toy yang extends Goods dan implements Taxable (15 menit) Untuk membuat hasil dapat diuji, gunakan asumsi operasional pada modul ini: pajak = price x taxRate.

```
public class Toy extends Goods implements Taxable {
private int minimumAge;

public Toy(String description, double price, int minimumAge) {
super(description, price);
this.minimumAge = minimumAge;
}
```



```
public int getMinimumAge() {
return minimumAge;
}
```

```
@Override
public double calculateTax() {
return getPrice() * taxRate;
}

@Override
public void display() {
super.display();
System.out.println("Minimum age : " + minimumAge);
System.out.println("Tax : " + calculateTax());
}
}
```

Checkpoint 9

Pada deklarasi `Toy extends Goods implements Taxable`, bagian mana yang menyatakan "Toy adalah Goods" dan bagian mana yang menyatakan "Toy memenuhi kontrak Taxable"?

Step 14 - Buat Book yang extends Goods dan implements Taxable (15 menit)

```
public class Book extends Goods implements Taxable {
private String author;

public Book(String description, double price, String author) {
super(description, price);
this.author = author;
}

public String getAuthor() {
return author;
}

@Override
public double calculateTax() {
return getPrice() * taxRate;
}

@Override
public void display() {
super.display();
System.out.println("Author : " + author);
System.out.println("Tax : " + calculateTax());
}
}
```

- Bandingkan Toy dan Book: keduanya punya parent yang sama dan contract yang sama, tetapi state spesifiknya berbeda.

Step 15 - Uji hierarchy dan contract secara utuh (15 menit)

```
public class GoodsTest {
public static void main(String[] args) {
Food food = new Food("Bread", 15000, 250);
Toy toy = new Toy("Robot", 120000, 8);


Book book = new Book("Java Basics", 90000, "A. Programmer");

food.display();
```

```
System.out.println();
toy.display();
System.out.println();
book.display();

System.out.println();
Taxable taxableToy = toy;
Taxable taxableBook = book;
System.out.println("Toy tax : " + taxableToy.calculateTax());
System.out.println("Book tax : " + taxableBook.calculateTax());
}
}
```

• Run dan simpan output.
• Perhatikan bahwa Food tidak diperlakukan sebagai Taxable pada model ini, sedangkan Toy dan Book dapat direferensikan sebagai Taxable.

Step 16 - Sengaja langgar kontrak interface (10 menit)

Di Toy.java, hapus atau comment sementara method calculateTax(), tetapi tetap pertahankan `implements Taxable`.

• Compile dan simpan pesan error.
• Jelaskan apa yang diminta compiler dari Toy.
• Kembalikan calculateTax() ke kondisi benar dan pastikan seluruh project kembali compile.

Checkpoint 10

Keyword implements bukan sekadar label. Ketika sebuah class implements interface, compiler menuntut class konkret tersebut memenuhi method yang dideklarasikan interface.

Step 17 - Bandingkan abstract class dan interface dari pengalaman code (5 menit)

Lengkapi tabel berikut berdasarkan apa yang benar-benar Anda alami, bukan dengan menyalin definisi.

Dapat dibuat object langsung?		
Menggunakan extends atau implements?		
Dapat mempunyai implementasi method konkret?		
Peran utamanya pada kasus praktikum?		
Apakah class dapat mempunyai parent class lain sekaligus?		

Step 18 - Uji sebelum menyatakan selesai (10 menit)

T5-01	new Sortable() dicoba	Compiler menolak instansiasi abstract class
T5-02	Employee extends Sortable tanpa compare()	Compiler menuntut implementasi abstract method
T5-03	Employee mengimplementasikan compare()	Hasil perbandingan -1/0/1 sesuai salary
T5-04	Manager hanya extends Employee	Manager tetap memiliki compare() melalui inheritance chain
T5-05	Manager extends Employee extends Sortable	Compiler menolak multiple extends class
T5-06	Toy/Book implements Taxable	calculateTax() dapat dipanggil
T5-07	calculateTax() dihapus dari Toy	Compiler menolak class konkret yang tidak memenuhi contract
T5-08	Food diuji sebagai Goods	State/behavior Goods tersedia melalui inheritance

5. Laporan dan Pengumpulan

5.1 Isi Laporan PDF

• Identitas: nama, NIM, kelas.

• Bagian A - Employee, Manager, Sortable: diagram hierarchy, code inti, output EmployeeTest, bukti abstract class tidak dapat diinstansiasi, bukti Employee wajib mengimplementasikan compare(), hasil compare(), dan bukti inheritance chain Manager.

• Bagian B - Goods dan Taxable: diagram hierarchy + interface, code inti Goods/Food/Toy/Book/Taxable, output GoodsTest, bukti contract interface ketika calculateTax() dihapus, dan hasil perhitungan tax. • Tabel skenario uji T5-01 s.d. T5-08 dengan status PASS/FAIL dan bukti.

• Jawaban refleksi pada Bagian 6.3.

5.2 Format Bukti

• Potongan code harus terbaca; jangan hanya screenshot IDE satu layar penuh.

• Output/error penting diberi caption singkat: apa yang diuji dan apa kesimpulannya.

• Jika ada error eksperimen, simpan bukti sebelum code diperbaiki.

• Pastikan source final sama dengan source yang dijelaskan pada PDF.

5.3 Struktur Folder Pengumpulan

```
NIM_Nama_PBO5/
├── Laporan_PBO5.pdf
```

```
└── source/
├── Sortable.java
├── Employee.java
├── Manager.java
```



```
├── EmployeeTest.java
├── Goods.java
├── Food.java
├── Taxable.java
├── Toy.java
├── Book.java
└── GoodsTest.java
```

Kumpulkan PDF laporan dan source code sesuai mekanisme yang ditetapkan dosen.

6. Checkpoint dan Refleksi

6.1 Definition of Done

1. Semua Step 1-18 telah dicoba, bukan hanya code final disalin.
2. Minimal dua bukti error eksperimen tersimpan: abstract class/interface tidak dapat diinstansiasi atau contract tidak dipenuhi.
3. Saya dapat menjelaskan inheritance chain Sortable <- Employee <- Manager.
4. Saya dapat menjelaskan mengapa Employee harus mengimplementasikan compare(). 5. Saya dapat menunjukkan perbedaan extends dan implements dari code.
6. Saya dapat menjelaskan mengapa Toy dan Book Taxable sedangkan Food tidak pada model yang diberikan. 7. Project final compile dan seluruh skenario utama PASS.
8. PDF dan source siap dikumpulkan.

6.2 Self Assessment

Saya memahami mengapa abstract class tidak dapat diinstansiasi.	
Saya dapat membuat abstract method dan implementasinya di subclass.	
Saya memahami inheritance chain tanpa menulis extends lebih dari satu class.	
Saya dapat membuat interface dan class yang implements interface tersebut.	
Saya memahami bahwa implements berarti memenuhi contract method.	
Saya dapat membedakan kapan konsep lebih cocok menjadi abstract class dan kapan menjadi interface.	

6.3 Refleksi - Jawab dari pengalaman praktikum

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat

compare()? 3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis

extends Sortable?

4. Mengapa `class Manager extends Employee extends Sortable` tidak diperlukan dan tidak valid di

Java? 5. Apa yang dijanjikan Toy ketika menulis `implements Taxable`?

6. Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

7. Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

8. Lengkapi dengan bahasa Anda sendiri: "Abstract class membantu saya ketika ..., sedangkan interface membantu saya ketika ..."